
Fisher-Guided Progressive Parameter Selection for Adaptive Fine-Tuning

Ghodsiyeh Rostami
Concordia University
rose.rostami@mail.concordia.ca

Po-Han Chen
Concordia University
pohan.chen@concordia.ca

Mahdi S. Hosseini
Concordia University
Mila - Québec AI Institute
mahdi.hosseini@concordia.ca

Abstract

Parameter-efficient fine-tuning (PEFT) aims to adapt pretrained models with a small trainable parameter subset, however, most existing methods choose this subset from fixed architectural heuristics rather than using dynamic, task-aware criteria. We introduce **FisherAdapTune**, a Fisher-guided Adaptive Fine-Tuning framework that progressively selects parameter groups by tracking the temporal drift of their Fisher geometry. Starting from a PAC-Bayesian view of fine-tuning, we decompose the generalization error bound into Fisher-weighted update costs and show that parameter groups whose curvature contribution has stabilized can be frozen to reduce the error bound without interrupting the remaining adaptation dynamics. FisherAdapTune formulates this criterion with a scale-invariant Jensen-Shannon distance between consecutive Fisher distributions, yielding an adaptive active parameter set. We evaluate our approach on a downstream segmentation task, and results show FisherAdapTune improves the in-distribution performance and zero-shot transfer in multiple settings, validating that Fisher structural drift is a useful signal for efficient, task-aware adaptation. We release our code publicly to enable further application of our proposed approach.

1 Introduction

Adapting pretrained deep neural networks to downstream tasks is a central problem in modern machine learning. While large-scale pretraining enables models to learn rich and transferable representations, leveraging these representations under limited data, distribution shifts, and computational constraints remains challenging [52, 27]. In practice, downstream environments are dynamic: data distributions evolve, tasks change, and models must be updated frequently. This makes efficient and robust adaptation essential for scalable deployment, especially when retraining from scratch is infeasible. Full fine-tuning, where all parameters are updated for the target task, is computationally expensive and memory-intensive, especially for large models, and can lead to overfitting or degradation in generalization under low-data regimes [46]. Moreover, recent studies suggest that classical scaling laws do not directly transfer to fine-tuning settings, with performance depending on complex interactions between model size, pretraining, and data regime [40, 29, 36]. These observations highlight the need for principled adaptation strategies beyond naïve full fine-tuning.

To address these challenges, *parameter-efficient fine-tuning* (PEFT) methods aim to reduce the number of trainable parameters while maintaining performance. Existing approaches include: (i) *reparameterization* methods such as LoRA, which constrain updates to low-rank subspaces [42, 18, 47]; (ii) *prompt-based* methods that introduce learnable tokens or embeddings [5, 22, 23]; (iii) *additive* methods that insert lightweight modules such as adapters [20, 7, 8, 24]; and (iv) *selective*

Table 1: Comparison of PEFT categories. ¹Parameter Efficiency, ²Task-Aware Configuration, ³Architectural Modifications, ⁴Theory-Grounded, ⁵Generalization, ⁶Inference Overhead.

Method	Param. Eff. ¹	Task-Awar. ²	Arch. Mod. ³	Theory. ⁴	Gen. ⁵	Inf. OH ⁶
Full Fine-Tuning	✗	✗	✗	✗	✗	✗
Reparameterization (LoRA [21])	✓	✗	✗	✓	✗	✗
Selective (BitFit [45])	✓	✗	✗	✗	✗	✗
Prompt Tuning (VPT [5])	✓	✗	✓	✗	✗	✓
Adapter-based (AdaptFormer [7])	✓	✗	✓	✗	✗	✓
Ours (FisherAdapTune)	✓	✓	✗	✓	✓	✗

methods that update predefined parameter subsets, such as biases (BitFit) [45] or normalization layers [48]. As summarized in Table 1, these methods reduce computational cost and can achieve competitive performance. However, they largely rely on heuristic or architecture-driven parameter selection, with fixed *a priori* choices that ignore dynamic, task-aware criteria and the evolving importance of parameters during training. This can lead to underutilization of critical components or unnecessary updates of redundant ones, and often requires extensive empirical tuning to identify an effective strategy. While a few works have investigated data-driven parameter selection [50], a principled, adaptive framework for identifying task-relevant parameters remains an open challenge.

In this work, we propose **FisherAdapTune**, a Fisher-guided Adaptive Fine-Tuning framework for dynamic parameter selection during fine-tuning. From a PAC-Bayesian perspective, we theoretically show that freezing parameters with stabilized contributions effectively tightens generalization bounds. Building upon this, our approach is grounded in the temporal evolution of the Fisher Information Matrix (FIM), which captures local curvature and parameter sensitivity. Instead of relying on static heuristics, we measure the *Jensen-Shannon (JS) distance* between successive Fisher distributions to track how parameter groups evolve during training (Figure 1). This provides a data-driven signal of *adaptation*: parameters undergoing significant structural change actively contribute to task learning, while those that stabilize can be progressively frozen. This yields a dynamic selective fine-tuning strategy that continuously adjusts the trainable parameter set throughout optimization. By leveraging Fisher *structural drift* rather than magnitude alone, FisherAdapTune identifies task-relevant parameters in a principled manner and avoids unnecessary updates. We validate FisherAdapTune on a domain-specific dense prediction task, demonstrating that it achieves competitive or superior performance compared to existing PEFT baselines. Furthermore, our approach improves robustness under distribution shifts, highlighting the importance of adaptive, task-aware parameter selection for generalization. In summary of our contributions, we

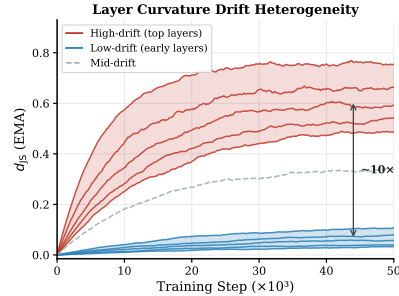


Figure 1: Layer-wise Fisher drift during full fine-tuning is uneven, motivating FisherAdapTune’s progressive freezing of stabilized layers.

- theoretically show that Fisher dynamics relate to generalization, explaining why freezing stabilized parameters improves generalization.
- therefore, introduce a principled, task-aware parameter selection criterion based on Fisher dynamics, using Jensen-Shannon distance to identify parameters that actively contribute to adaptation.
- propose **FisherAdapTune**, an adaptive, model-agnostic fine-tuning framework that dynamically adjusts the trainable parameter set during training without heuristic design.
- validate FisherAdapTune on a downstream task across model families, demonstrating superior performance over existing PEFT baselines while reducing the effective number of trainable parameters and improving zero-shot generalization.

2 Related Work

Parameter-Efficient Fine-Tuning (PEFT) aims to adapt pretrained models with reduced computational and memory cost while maintaining performance [46, 17]. Selective fine-tuning methods, inspired in part by the Lottery Ticket Hypothesis [13], update only a subset of parameters to mitigate overfitting and improve efficiency [1]. Existing approaches identify important parameters using heuristics such as magnitude of updates (e.g., LT-SFT [4]), architectural choices (e.g., LayerNorm tuning [48]), or probabilistic criteria (e.g., BayesTune [25]), while others rely on learnable masks

(Masking [50], Diff-Pruning [16]) or gradient dynamics (AutoFreeze [30]) to guide selection. Despite promising empirical results, these methods largely depend on heuristic signals and lack a principled, theoretically grounded mechanism for identifying task-relevant parameters. On the other hand, **Fisher-guided methods** leverage the Fisher Information Matrix (FIM) to estimate parameter importance based on loss sensitivity [26, 35]. Prior work such as Child-Tuning [44] and FISH [39] uses diagonal Fisher approximations to select sparse subsets of parameters, while more recent approaches such as FisherTune [49] incorporate both task and domain sensitivity via extended Fisher formulations. However, these methods typically rely on static or computationally expensive Fisher estimates and introduce additional complexity or hyperparameters, limiting scalability. Overall, a principled and efficient framework that adaptively identifies task-relevant parameters during training remains an open challenge.

3 Methodology

We propose **FisherAdapTune**, a task-aware fine-tuning framework that dynamically selects which parameter groups to update. Instead of fixing the trainable subset before training, FisherAdapTune tracks the temporal evolution of Fisher information, a tractable approximation for local curvature, and progressively freezes stabilized groups. This section formalizes the Fisher-based adaptation signal and the resulting adaptive selection procedure.

3.1 Role of Fisher Information in Generalization and Adaptation

We revisit Probably Approximately Correct (PAC)-Bayesian learning theory [33, 34, 12, 51, 31] to derive a principled basis for fine-tuning. Let P denote a prior distribution over model parameters, instantiated by the pretrained weights of the foundation model, and let Q_T denote the posterior after T steps of fine-tuning. The PAC-Bayes theorem states that, with probability at least $1 - \delta$ over the training sample, the generalization error satisfies:

$$\mathbb{E}_{Q_T}[\mathcal{L}_{\text{ood}}] \leq \mathbb{E}_{Q_T}[\mathcal{L}_{\text{in}}] + \sqrt{\frac{D_{\text{KL}}(Q_T \| P) + \log(N/\delta)}{2N}}, \quad (1)$$

where N is the number of training samples. The bound is governed by $D_{\text{KL}}(Q_T \| P)$ where excessive deviation from the pretrained prior loosens the guarantee [31]. To relate this divergence to parameter updates, we consider how much a small change in parameters shifts the model’s output distribution. Let $p_{\theta}(\mathbf{y} \mid \mathbf{x})$ denote the predictive distribution over $\mathbf{y}$ produced by a model with parameter vector $\theta \in \mathbb{R}^{|\theta|}$ given input $\mathbf{x}$. A natural measure of the effect of a perturbation $\delta \in \mathbb{R}^{|\theta|}$ on the model’s predictions is $D_{\text{KL}}(p_{\theta}(\mathbf{y} \mid \mathbf{x}) \| p_{\theta+\delta}(\mathbf{y} \mid \mathbf{x}))$. It can be shown [11, 3, 38] that as $\|\delta\| \rightarrow 0$, a second-order expansion averaged over the input distribution yields:

$$\mathbb{E}_{\mathbf{x}}[D_{\text{KL}}(p_{\theta+\delta} \parallel p_{\theta})] = \frac{1}{2} \delta^{\top} \mathbf{F}_{\theta} \delta + \mathcal{O}(\|\delta\|^3), \quad (2)$$

where $\mathbf{F}_{\theta}$ is the Fisher Information Matrix (FIM) [2], $\mathbf{F}_{\theta} = \mathbb{E}_{\mathbf{x}} [\mathbb{E}_{\mathbf{y} \sim p_{\theta}(\mathbf{y} \mid \mathbf{x})} [\nabla_{\theta} \log p_{\theta}(\mathbf{y} \mid \mathbf{x}) \nabla_{\theta} \log p_{\theta}(\mathbf{y} \mid \mathbf{x})^{\top}]]$, capturing the local curvature of the predictive distribution. To connect the PAC-Bayes divergence to per-step Fisher costs, we adopt a Laplace approximation in which all weight-space posteriors share the pretrained Fisher as their common precision: $Q_t = \mathcal{N}(\theta_t, \mathbf{F}_{\theta_0}^{-1})$ for $t = 0, \dots, T$, with $Q_0 = P$. This anchors the posterior geometry at the pretrained initialization, making $D_{\text{KL}}(Q_T \| P)$ a direct measure of accumulated parameter drift from the prior.

Assumption 3.1 (Slowly varying Fisher). The overall Fisher scale varies slowly during fine-tuning: $\|\mathbf{F}_{\theta_t}\|_F \approx \|\mathbf{F}_{\theta_0}\|_F$ uniformly over $t \in \{0, \dots, T-1\}$. This holds when the model is updated with a small learning rate from a pretrained stationary point. Crucially, this assumption constrains only the Fisher *magnitude*; the normalized shape $\mathbf{P}_{\ell}^t = \mathbf{F}_{\theta_{t,\ell}} / \|\mathbf{F}_{\theta_{t,\ell}}\|_F$ is free to reorient across layers and iterations as the importance mass may redistribute across parameters while the layer’s aggregate curvature stays constant.

Proposition 3.2. Let $Q_0 = P = \mathcal{N}(\theta_0, \mathbf{F}_{\theta_0}^{-1})$ denote the pretrained prior, and let all posteriors share the pretrained precision: $Q_t = \mathcal{N}(\theta_t, \mathbf{F}_{\theta_0}^{-1})$. Then the following identity holds (proof in Appendix A.1):

$$D_{\text{KL}}(Q_T \| P) = \sum_{t=0}^{T-1} \left[\frac{1}{2} \delta_t^{\top} \mathbf{F}_{\theta_0} \delta_t + \delta_t^{\top} \mathbf{F}_{\theta_0} (\theta_t - \theta_0) \right]. \quad (3)$$

Under L2 regularization (weight decay), the update takes the form $\delta_t = -\eta(\nabla\mathcal{L}(\theta_t) + \lambda(\theta_t - \theta_0))$, so each cross term contains the strictly negative component $-\eta\lambda\|\theta_t - \theta_0\|_{\mathbf{F}_{\theta_0}}^2 \leq 0$, making the quadratic sum an upper bound on $D_{\text{KL}}(Q_T\|P)$. Without regularization, the cross terms are negligible when the accumulated displacement $\|\theta_t - \theta_0\|$ is small; as is enforced by the adaptation.

Generalization bound governed by FIM

In both cases, invoking Assumption 3.1 to replace $\mathbf{F}_{\theta_0}$ by $\mathbf{F}_{\theta_t}$, we obtain the following bound:

$$D_{\text{KL}}(Q_T\|P) \lesssim \sum_{t=0}^{T-1} \frac{1}{2} \delta_t^\top \mathbf{F}_{\theta_t} \delta_t, \quad (4)$$

where $\lesssim$ denotes a inequality under weight decay and an approximation otherwise.

Equation (4) shows that the total divergence from the pretrained prior decomposes into a sum of curvature-weighted updates along the optimization trajectory. Each step contributes a quadratic form governed by the local Fisher Information Matrix, capturing the geometry of the loss landscape. Consequently, the KL divergence, and therefore the PAC-Bayes generalization bound in Eq. (1), is controlled by the accumulation of these Fisher, weighted increments. This establishes the Fisher Information as a principled, geometry-aware measure of how parameter updates impact generalization. Importantly, this formulation shifts the perspective from parameter magnitude to *curvature-aligned change*, providing a theoretical foundation for using Fisher dynamics to identify which parameters meaningfully contribute to task adaptation. The remaining question is how to exploit this Fisher-governed upper bound to localize the most informative layers for adaptation.

3.2 Curvature Shift as an Adaptation Signal

To make this criterion actionable, we decompose the Fisher-governed KL cost across layers and use the resulting curvature shifts as a proxy for adaptation relevance.

Assumption 3.3 (Layer-wise Fisher independence). The FIM is block-diagonal across L layers [32]: $\mathbf{F}_\theta = \text{diag}(\mathbf{F}_{\theta_1}, \dots, \mathbf{F}_{\theta_L})$.

Definition 3.4 (Per-layer KL cost). The per-iteration per-layer contribution to the approximated KL is $c_{t,\ell} := \delta_{t,\ell}^\top \mathbf{F}_{\theta_{t,\ell}} \delta_{t,\ell}$, and its cumulative contribution is $\mathcal{C}_\ell := \sum_{t=0}^{T-1} c_{t,\ell}$.

Proposition 3.5 (Layer-wise decomposition of the KL approximation). Under Assumptions 3.1 and 3.3, the approximation in Eq. (4) decomposes as:

$$D_{\text{KL}}(Q_T\|P) \approx \frac{1}{2} \sum_{\ell=1}^L \mathcal{C}_\ell. \quad (5)$$

Since each $c_{t,\ell} = \delta_{t,\ell}^\top \mathbf{F}_{\theta_{t,\ell}} \delta_{t,\ell} \geq 0$ (a quadratic form under a positive semi-definite matrix), every term contributes non-negatively to the approximated KL.

Corollary 3.6 (Freezing stabilized layers reduces the approximated KL). If layer ℓ is frozen at iteration t^* , the approximated KL in Eq. (5) is reduced by $\Delta\mathcal{C}_\ell = \frac{1}{2} \sum_{t=t^*}^{T-1} c_{t,\ell} \geq 0$, since each $c_{t,\ell} \geq 0$.

Corollary 3.6 shows that freezing a layer reduces the approximated KL, and hence the generalization bound, by removing a non-negative contribution. However, freezing may also suppress task adaptation. Therefore, the key question is *which layers can be frozen without removing essential adaptation dynamics*. We posit that layers whose contributions have effectively *stabilized* can be frozen without compromising performance. Importantly, stabilization should be understood in terms of the *evolution* of the contribution, rather than its absolute value. If a layer's contribution changes only marginally across iterations, $c_{t,\ell} - c_{t-1,\ell} \approx 0$, then it no longer induces meaningful updates to the generalization cost, indicating that its role in ongoing adaptation has saturated. To characterize such behavior, we define the per-layer contribution drift:

$$c_{t,\ell} - c_{t-1,\ell} = \delta_{t,\ell}^\top \mathbf{F}_{\theta_{t,\ell}} \delta_{t,\ell} - \delta_{t-1,\ell}^\top \mathbf{F}_{\theta_{t-1,\ell}} \delta_{t-1,\ell}.$$

Assumption 3.7 (Smooth parameter updates). Consecutive parameter updates are approximately equal: $\delta_{t,\ell} \approx \delta_{t-1,\ell}$ for all layers ℓ and iterations t . This is satisfied in fine-tuning settings with small learning rates or momentum-based optimizers (e.g., Adam), where gradient directions vary smoothly and the model operates in a locally flat region of the pretrained loss landscape.

Adaptation governed by FIM drift (Under Assumption 3.7)

$$c_{t,\ell} - c_{t-1,\ell} \approx \boldsymbol{\delta}_{t,\ell}^\top \Delta \mathbf{F}_{t,\ell} \boldsymbol{\delta}_{t,\ell}, \quad \Delta \mathbf{F}_{t,\ell} := \mathbf{F}_{\boldsymbol{\theta}_{t,\ell}} - \mathbf{F}_{\boldsymbol{\theta}_{t-1,\ell}}. \quad (6)$$

Equation (6) shows that the evolution of a layer’s contribution to the approximated generalization bound is governed by the *temporal variation of its Fisher Information*. When the Fisher stabilizes, $\mathbf{F}_{\boldsymbol{\theta}_{t,\ell}} \approx \mathbf{F}_{\boldsymbol{\theta}_{t-1,\ell}}$, the contribution drift $c_{t,\ell} - c_{t-1,\ell}$ becomes negligible: the layer no longer induces meaningful changes to the approximated generalization bound, and further updates yield diminishing adaptation returns while still incurring additional divergence cost. Freezing such layers prevents unnecessary accumulation in the KL, and hence in the PAC Bayes bound, while preserving the adaptation of layers that remain active. Conversely, layers with significant Fisher drift continue to reshape their local curvature and contribute meaningfully to task adaptation, which should remain trainable.

3.3 Fisher-Guided Adaptive Selective Fine-Tuning Algorithm (FisherAdapTune)

A Scale-Invariant Measure of Fisher Structural Drift. The preceding analysis motivates the temporal variation of the FIM as a principled signal for parameter selection. To design a criterion that captures structural changes in Fisher information while remaining invariant to scale, we map the Fisher tensor at iteration t and layer ℓ to a probability distribution via log-normalization: $p_{t,\ell}(i) = \frac{\log(1+F_{\ell}^t(i))}{\sum_j \log(1+F_{\ell}^t(j))}$. This transformation mitigates the effect of large-magnitude entries while preserving the relative structure of the Fisher spectrum. To quantify the evolution of this distribution across training iterations, we measure the divergence between consecutive Fisher-induced distributions using the Jensen-Shannon (JS) divergence. For two probability mass functions p and q , the JS divergence is defined as:

$$\text{JS}(p, q) = \frac{1}{2} \text{KL}(p \parallel M) + \frac{1}{2} \text{KL}(q \parallel M), \quad M = \frac{1}{2}(p + q). \quad (7)$$

And the JS distance is:

$$d_{\text{JS}}(p, q) = \sqrt{\text{JS}(p, q)}. \quad (8)$$

which yields a symmetric and bounded metric. Full computation details are provided in Appendix A.2.

Proposition 3.8 (JS distance operationalizes structural Fisher drift). *Let $p_{t,\ell}$ and $p_{t-1,\ell}$ denote the log-normalized Fisher distributions defined in Appendix A.2. The Jensen-Shannon distance $d_{\text{JS}}(p_{t,\ell}, p_{t-1,\ell})$ satisfies two key properties:*

1. *Structural sensitivity: $d_{\text{JS}}(p_{t,\ell}, p_{t-1,\ell}) = 0$ if and only if the log-normalized Fisher distributions coincide, i.e., the Fisher’s relative structure, the normalized shape tensor $\mathbf{P}_{\ell}^t = \mathbf{F}_{\ell}^t / \|\mathbf{F}_{\ell}^t\|_F$, has not changed.*
2. *Scale invariance: d_{JS} is invariant to uniform scaling $\mathbf{F}_{\ell}^t \rightarrow \alpha \mathbf{F}_{\ell}^t$ for any $\alpha > 0$, since the log-normalization step removes the overall magnitude.*

Consequently, d_{JS} is identically zero under pure magnitude drift ($\mathbf{P}_{\ell}^t = \mathbf{P}_{\ell}^{t-1}$) and positive whenever the curvature structure has reoriented. This directly operationalizes the adaptation signal in Eq. (6): the layers for which $d_{\text{JS}}(p_{t,\ell}, p_{t-1,\ell}) \approx 0$ are precisely those for which $\mathbf{F}_{\boldsymbol{\theta}_{t,\ell}} \approx \mathbf{F}_{\boldsymbol{\theta}_{t-1,\ell}}$, and hence whose contribution drift $c_{t,\ell} - c_{t-1,\ell}$ is negligible.

Empirical Validation: Fisher structural drift is heterogeneous across layers. Proposition 3.8 establishes that the Jensen-Shannon (JS) distance between consecutive Fisher distributions captures structural curvature drift, and therefore serves as a signal for adaptation. To validate this, we track $d_{\text{JS}}(p_{t,\ell}, p_{t-1,\ell})$ throughout full fine-tuning across SAM2 [37] and SegFormer [43] architectures. We adopt the diagonal Kronecker approximation proposed by AdaFisher [15] as an efficient approximation of FIM:

$$\tilde{\mathbf{F}}_{\ell}^D = \mathbf{H}_{D_{\ell-1}} \otimes \mathbf{S}_{D_{\ell}}, \quad (9)$$

where $\mathbf{H}_{D_{\ell-1}}$ and $\mathbf{S}_{D_{\ell}}$ denote diagonal activation and gradient correlation factors, respectively. This approximation reduces computational and memory complexity to linear in the number of parameters. Figure 2 shows the evolution of d_{JS} across all layers during SAM2 fine-tuning. The heatmap (Panel a) reveals a clear distinction between high-drift and near-stationary layers, with an $11\times$ spread in mean drift. Panel (b) shows that this separation emerges early ($\sim 5\text{k}$ steps) and persists throughout training, while distributional analysis (Panel c) shows systematic differences across parameter

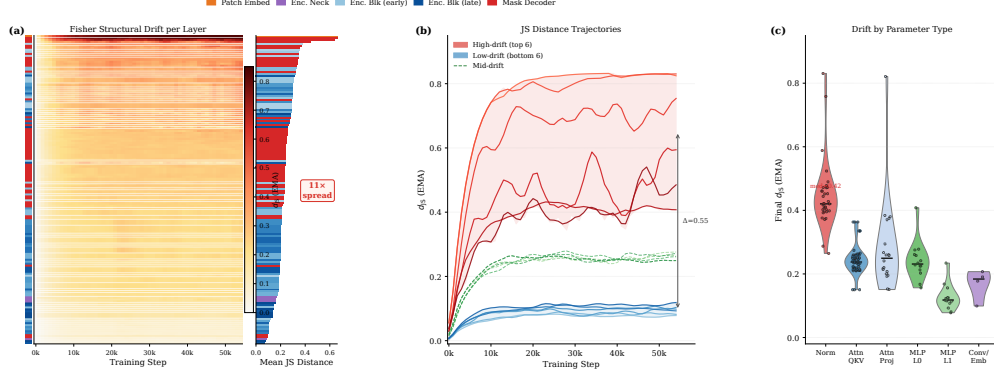


Figure 2: Layer-wise Jensen-Shannon (JS) distance between consecutive Fisher distributions in SAM2-Tiny during full fine-tuning on crack segmentation; $EMA[d_{JS_i}(p(\tilde{\mathbf{F}}_i^{t-1}), p(\tilde{\mathbf{F}}_i^t))]$. types. This empirical heterogeneity validates Proposition 3.8 where layers with $d_{JS} \approx 0$ exhibit negligible curvature drift and therefore negligible contribution drift (Eq. 6), while persistently high d_{JS} identifies layers undergoing active curvature reorientation and continued adaptation. Importantly, this heterogeneity is not only layer-wise but also *intra-layer*. Figure 7 extends this analysis to column-wise parameter groups. Within-layer variation is substantial, with drift differences exceeding 0.5 in some layers and remaining above 0.1 for a large fraction of the network. These patterns emerge early and persist, indicating that adaptation is both layer-dependent and direction-dependent. Similar patterns were observed in SegFormer, which is illustrated in Figure 8 in Appendix B.1.

Implication: Fisher Drift Enables Selective Adaptation

Fisher structural drift is non-uniform across both layers and parameter groups. This implies that only a subset of parameters remains dynamically relevant during fine-tuning. Consequently, d_{JS} provides a principled criterion for selective freezing where parameters with negligible drift can be safely frozen, while those with persistent curvature change should remain trainable.

FisherAdapTune. Building upon the implication above, we propose *FisherAdapTune*, a Fisher-guided Adaptive Fine-Tuning framework that updates only parameter groups that remain useful for adaptation. The motivation is the non-uniform *Fisher structural drift* observed across layers and parameter blocks (Figure 7) where some groups continue to reorient their local curvature, while others quickly stabilize. FisherAdapTune tracks this evolution with the Jensen-Shannon (JS) distance between consecutive Fisher distributions. Groups with low stable JS distance are treated as saturated and progressively frozen, where groups with persistent drift remain trainable. All parameters are trainable at initialization, which creates an implicit warm-up phase before the first freezing decision. Fisher statistics are collected every m steps and smoothed over time, while freezing is evaluated only every n steps.

Algorithm 1: FisherAdapTune Algorithm

Require: Pre-Trained Model θ , dataset $\mathcal{D}$, hyperparameters: EMA factor β , threshold scale λ , block count k , fisher collection interval m , freezing interval $n \geq 0.02T_{\max}$, total steps $T_{\max}$

- 1: **for** $t = 1, \dots, T_{\max}$ **do**
- 2: Compute $\nabla_{\theta} \mathcal{L}(\mathcal{D}; \theta)$ via forward/backward pass
- 3: **if** $t \bmod m = 0$ **then**
- 4: Estimate per-layer Fisher curvature $\tilde{\mathbf{F}}_{D_i}^t$ via Eq. 9
- 5: Partition $\tilde{\mathbf{F}}_{D_i}^t$ column-wise into k blocks $\{\tilde{\mathbf{F}}_{w_{ik}}^t\}_{k=1}^K$
- 6: Build PMFs $p(\tilde{\mathbf{F}}_{w_{ik}}^t)$ via Eq. 14
- 7: Measure curvature shift $d_{JS_{ik}}^t(p_{t-1}, p_t)$ via Eq 8
- 8: Smooth via EMA: $\hat{d}_{JS_{ik}}^t \leftarrow \beta \hat{d}_{JS_{ik}}^{t-1} + (1 - \beta) d_{JS_{ik}}^t$
- 9: Compute mean curvature shift $\bar{d}_{JS_{ik}}$
- 10: **if** $t \bmod n = 0$ **then**
- 11: Compute global statistics of JS distances $\mu \leftarrow \text{mean}_{i,k}(\bar{d}_{JS_{ik}}); \sigma \leftarrow \text{std}_{i,k}(\bar{d}_{JS_{ik}})$
- 12: Set adaptive threshold $\tau \leftarrow \mu + \lambda\sigma$
- 13: **Freeze** stabilized parameter groups ($\bar{d}_{JS_{ik}} < \tau$)
- 14: **Update** only the remaining tunable parameter groups $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\mathcal{D}; \theta)$
- 15: **end for**

This separates noisy curvature estimation from parameter selection and prevents premature freezing. At each freezing point, FisherAdapTune applies an adaptive threshold over the smoothed JS scores, yielding a data-driven active set without manual layer schedules. Algorithm 1 summarizes the procedure; additional implementation details are provided in Algorithm 2 in Appendix C.

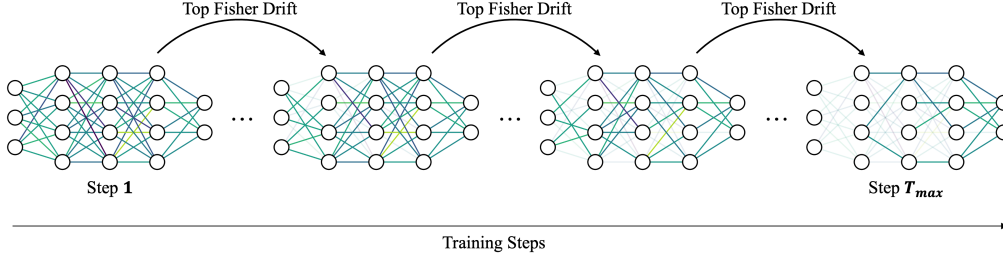


Figure 3: Schematic overview of FisherAdapTune. The algorithm periodically estimates Fisher information, measures Jensen-Shannon distance between consecutive Fisher distributions to identify stabilized parameter groups. Groups with high Fisher drift remain tunable, while stabilized groups are frozen.

4 Experiments & Results

Experimental Setup. We evaluate FisherAdapTune on crack segmentation, a challenging downstream task due to the curvilinear structure of cracks leading to distribution shift from pre-training data [10, 9]. We use OmniCrack30k [6], a 30K-image benchmark aggregated from more than 20 sub-datasets and spanning concrete, asphalt, ceramic, masonry, and steel surfaces under diverse imaging conditions. Models are trained on OmniCrack and evaluated both in-distribution and zero-shot on Concrete3k [28, 41], Facade390 [14], and Road420 [14]. More experimental details are provided in Appendix B.

Effective Number of Trainable Parameters. Since FisherAdapTune progressively changes the active parameter set, we report the average number of parameters updated per iteration $\bar{N} = \frac{\sum_{t=1}^T N_t}{T}$, where N_t is the number of trainable parameters at iteration t and T is the number of training iterations. For static PEFT methods that update the same number of parameters at each iteration, this equals the fixed trainable count; for FisherAdapTune it captures the cost of the full progressive-freezing trajectory rather than only the final active set.

Ablative Studies. We ablate the freezing threshold λ , which controls how conservatively FisherAdapTune retains parameter groups whose Fisher geometry is still changing, where a larger $|\lambda|$ keeps more groups trainable. The trends in Figure 4 based on Tables 3 and 4 in Appendix B.2, show a consistent parameter efficiency-adaptation trade-off. More conservative freezing generally improves OmniCrack (in-distribution) performance, while the best zero-shot behavior often occurs before retaining the largest number of parameters. This is evident for SegFormer-MiT-B3 (Table 4), where the optimal selected threshold performs best in both zero-shot and in-distribution results. Overall, across SAM2 and SegFormer, the best selected λ values provide balanced in- and out-of-distribution performances. FisherAdapTune preserves the parameter groups that remain useful for adaptation while freezing groups with stabilized Fisher geometry, mitigating over-specialization to the training set.

Comparative Results. We compare FisherAdapTune against full fine-tuning, LoRA, BitFit, LayerNorm tuning, decoder-only tuning, and random parameter selection. Random selection uses comparable parameter budgets and chooses parameters randomly from every layer (the best random selection). The comparative results in Table 2, summarized visually in Figures 5a and 5b, show that FisherAdapTune recovers most of the benefit of full fine-tuning with fewer effective trainable parameters. On SAM2, it remains close to full fine-tuning in-distribution and improves zero-shot averages for the large model, indicating that progressive freezing can improve robustness rather than merely reduce cost. On SegFormer, the advantage is stronger where FisherAdapTune matches or exceeds full fine-tuning in zero-shot generalization, especially for MiT-B3 variant, while using a much smaller active parameter budget. Random selection at matched budgets is consistently weaker, showing that the gains are not explained by parameter count alone. The results support the core design choice of FisherAdapTune: selecting parameters according to ongoing Fisher drift yields a more reliable efficiency-generalization trade-off than static PEFT rules or unguided sparse updates.

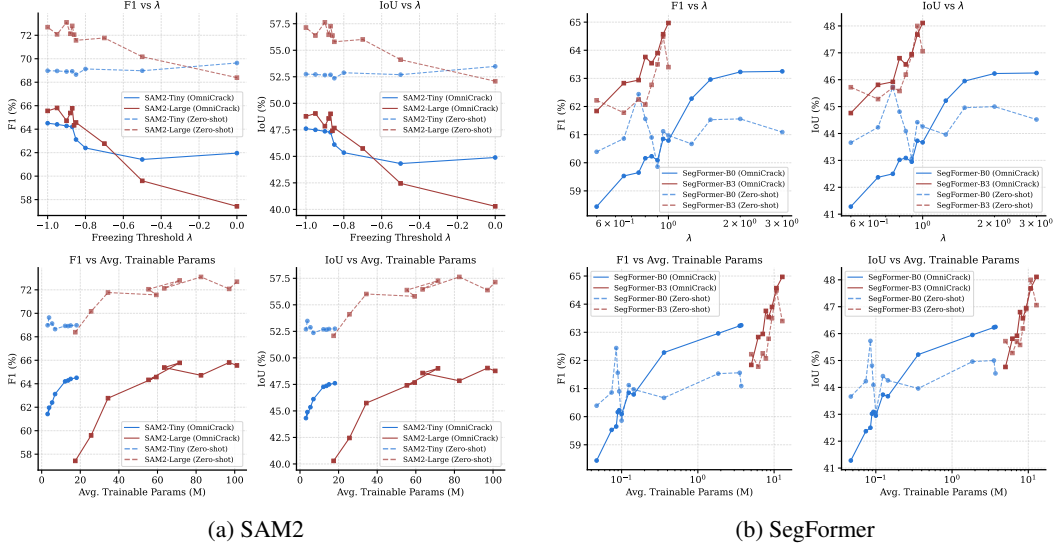


Figure 4: Ablation trends over the freezing threshold λ . FisherAdapTune balances in-distribution adaptation and zero-shot generalization by avoiding both premature freezing and excessive source-task specialization.

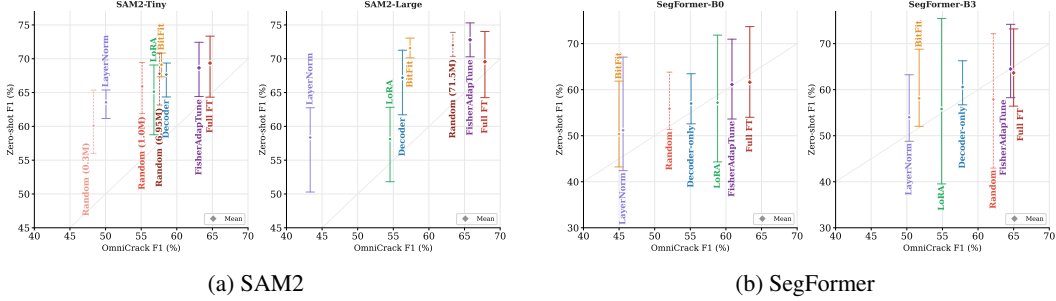


Figure 5: Comparative fine-tuning performance for SAM2 and SegFormer. OmniCrack F1-score measures in-distribution performance, while the zero-shot F1 measures average generalization under dataset shift. FisherAdapTune outperforms other PEFT methods, and shows its strongest gains in zero-shot evaluation.

What parameters get selected by FisherAdapTune? Figure 6 visualizes the trainable set chosen by FisherAdapTune over the course of fine-tuning for SAM2-Tiny and SegFormer-MiT-B3. Panels (i-a) and (ii-a) show *when* a parameter group is frozen, while Panels (i-b) and (ii-b) show *where* that group sits in the architecture. The resulting selection pattern is consistent across the two model families. FisherAdapTune freezes early feature-extraction components first, including low-level convolutional or patch-embedding groups and other parameters whose Fisher geometry stabilizes quickly after exposure to the training data. In contrast, deeper and more task-dependent modules remain active for longer: SAM2 retains adaptation in the image-encoder blocks, FPN-style neck components, prompt/mask-decoder-related groups, and output-facing parameters. SegFormer retains later MiT encoder blocks, MLP/attention projections, normalization groups that continue to shift, and decoder head parameters. This behavior is not imposed by a layer-depth rule or a hand-crafted PEFT template. It emerges from the Fisher-drift criterion where groups encoding generic edges, textures, and local contrast rapidly reach stable curvature structure, whereas groups responsible for semantic aggregation and mask formation continue to reorient their local curvature as the model learns the crack-segmentation mapping. Thus, *FisherAdapTune recovers the standard transfer-learning principle* that generic representations need less adaptation and task-dependent parameter groups should carry most of the fine-tuning, but does so automatically using a curvature-based criterion.

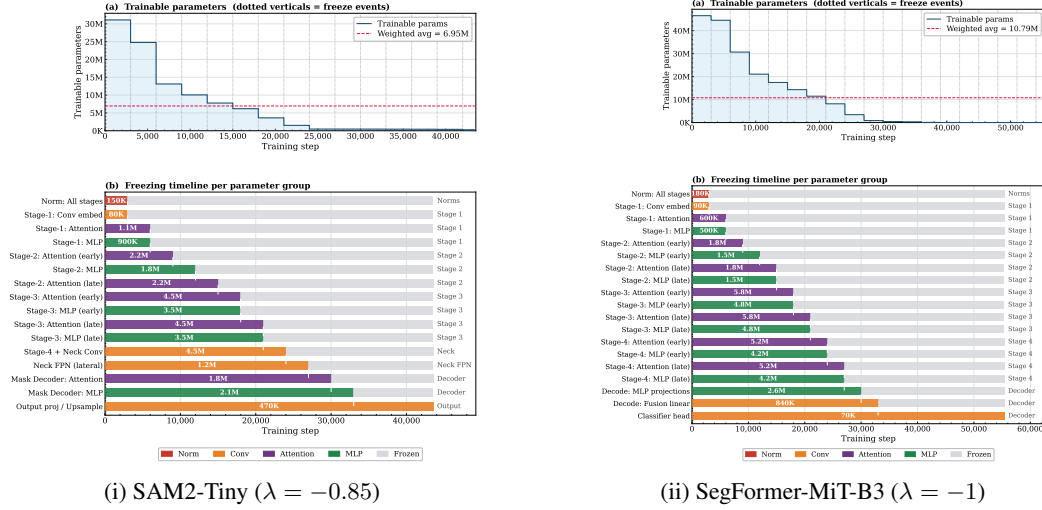


Figure 6: Parameter selection dynamics by FisherAdapTune. The algorithm progressively freezes stabilized generic components of models (e.g., shallow encoder blocks) and retains task-dependent groups longer (e.g., deeper encoder blocks and decoder head), automatically recovering a standard transfer-learning hierarchy using a curvature-aware criterion rather than a fixed criterion.

Table 2: Performance comparison of fine-tuning methods for crack segmentation in terms of in-distribution performance (OmniCrack test) and zero-shot generalization. Bold entries show the best per column within each model block; underline shows the second best. Effect. Params denotes number of Effective Tunable Parameters.

Model	Fine-tune Method	Effect. Params	OmniCrack		Zero-shot F1 (%)					Zero-shot IoU (%)			
			F1	IoU	Conc.	Fac.	Road	Avg.±Std		Conc.	Fac.	Road	Avg.±Std
SAM2-Tiny	Freeze	0.0M	4.37	2.24	8.21	4.13	4.47	5.60±2.27		4.28	2.11	2.28	2.89±1.21
	Full FT	38.9M	64.67	47.79	73.34	<u>70.39</u>	64.32	69.35±4.60		57.91	<u>54.31</u>	47.40	53.21±5.34
	LoRA	0.28M	56.79	39.65	69.07	67.51	58.76	65.11±5.56		52.75	50.96	41.60	48.44±5.99
	BitFit	0.11M	57.85	40.69	70.84	69.17	67.33	<u>69.11±1.76</u>		54.85	52.87	50.74	<u>52.82±2.05</u>
	LayerNorm	0.03M	50.06	33.39	64.15	65.38	61.16	<u>63.56±1.77</u>		47.23	48.56	44.05	46.61±1.89
	Decoder-only	4.2M	58.53	41.48	69.26	69.36	64.35	<u>67.66±2.86</u>		52.98	53.09	47.44	51.17±3.23
	Random	0.3M	48.30	31.84	56.00	65.36	58.93	60.09±3.91		38.88	48.54	41.77	43.07±4.05
	Random	1.0M	55.14	38.07	66.41	69.44	61.88	65.91±3.11		49.71	53.19	44.80	49.24±3.44
	Random	6.95M	57.57	40.42	69.41	70.83	63.15	67.80±3.34		53.15	54.83	46.14	51.38±3.76
SAM2-Large	FisherAdapTune	6.95M	<u>63.12</u>	<u>46.11</u>	<u>72.43</u>	69.10	<u>64.42</u>	<u>68.65±4.0</u>		<u>56.77</u>	52.79	<u>47.52</u>	<u>52.36±4.6</u>
	Freeze	0.0M	3.54	1.80	6.29	2.41	3.93	4.21±1.95		3.25	1.22	2.00	2.16±1.02
	Full FT	224M	67.86	51.35	74.03	64.25	70.39	69.56±5.0		58.77	47.33	54.31	53.47±5.8
	LoRA	0.221M	54.54	37.49	59.72	62.82	51.81	58.11±5.68		42.57	45.79	34.96	41.11±5.56
	BitFit	0.41M	57.35	40.20	73.05	70.13	<u>71.55</u>	<u>71.58±1.19</u>		<u>57.54</u>	54.00	<u>55.70</u>	55.75±1.44
	LayerNorm	0.12M	43.33	27.66	50.29	62.75	61.99	<u>58.34±5.70</u>		33.59	45.72	44.92	41.41±5.54
	Decoder-only	4.2M	56.28	39.16	71.26	68.61	61.71	67.19±4.93		55.35	52.22	44.62	50.73±5.52
	Random	71.49M	63.36	46.37	73.90	<u>70.35</u>	71.79	72.01±1.46		58.60	<u>54.26</u>	55.99	56.29±1.79
	FisherAdapTune	71.49M	<u>65.78</u>	<u>49.01</u>	72.81	75.30	70.29	<u>72.80±2.04</u>		57.24	60.38	54.19	57.27±2.53
SegFormer-MiT-B0	Full FT	3.71M	63.35	46.36	73.71	57.14	54.00	61.61±8.64		58.37	40.00	36.98	45.11±9.45
	LoRA	0.23M	58.83	41.68	71.85	55.38	44.31	57.18±11.31		56.06	38.29	28.46	40.94±11.42
	BitFit	0.02M	44.99	29.02	61.86	45.95	43.23	50.34±8.21		44.79	29.83	27.57	34.06±7.64
	LayerNorm	0.01M	45.56	29.50	67.07	44.07	42.41	51.18±11.25		50.46	28.26	26.91	35.21±10.79
	Decoder-only	0.40M	55.11	38.04	63.47	54.91	52.58	56.98±4.68		46.48	37.85	35.66	40.00±4.67
	Random	0.12M	52.08	35.21	63.81	51.33	52.45	55.87±5.64		46.85	34.53	35.55	38.98±5.58
SegFormer-MiT-B3	FisherAdapTune	0.12M	<u>60.85</u>	<u>43.73</u>	<u>70.97</u>	58.77	<u>53.63</u>	<u>61.12±7.27</u>		<u>55.01</u>	41.62	<u>36.64</u>	<u>44.42±7.75</u>
	Full FT	44.60M	64.99	48.14	<u>73.20</u>	61.31	56.42	63.64±7.04		<u>57.72</u>	44.20	39.30	47.07±7.78
	LoRA	1.79M	54.88	37.82	75.47	52.39	39.53	55.79±14.87		60.60	35.49	24.63	40.24±15.06
	BitFit	0.14M	51.73	34.89	68.79	52.04	53.53	58.12±7.56		52.42	35.17	36.55	41.38±7.82
	LayerNorm	0.05M	50.33	33.63	63.24	50.01	48.79	54.01±6.54		46.24	33.34	32.27	37.28±6.34
	Decoder-only	0.53M	57.83	40.67	66.28	56.71	58.72	60.57±4.12		49.57	39.57	41.57	43.57±4.32
SegFormer-MiT-B3	Random	10.73M	62.15	45.09	72.16	58.56	42.96	57.89±11.93		56.45	41.40	27.36	41.74±11.88
	FisherAdapTune	10.73M	<u>64.57</u>	<u>47.68</u>	74.19	<u>61.04</u>	<u>58.26</u>	<u>64.49±6.94</u>		58.97	<u>43.93</u>	<u>41.11</u>	48.00±7.83

5 Conclusion

We introduced FisherAdapTune, a Fisher-guided Adaptive Fine-Tuning framework that uses the temporal drift of Fisher information to decide which parameter groups should remain trainable. Unlike static PEFT methods, FisherAdapTune does not rely on hand-crafted layer choices or fixed adaptation rules. It freezes parameters once their curvature geometry has stabilized and preserves updates for groups that continue to drive task adaptation. Our theoretical analysis provides the main foundation for this criterion. From a PAC-Bayesian perspective, we relate Fisher geometry to the generalization error bound, showing why stabilized groups can be frozen without sacrificing the remaining adaptation dynamics. Empirically, experiments on a downstream crack segmentation task with two model families demonstrate a parameter efficiency-generalization trade-off and improvement on zero-shot transfer in multiple settings. These results suggest that Fisher drift is a useful signal for task-aware adaptation. Overall, this work provides both the theoretical foundation and the Fisher-drift criterion for adaptive fine-tuning. Building on this foundation, future work will extend the evaluation to broader architectures, datasets, and downstream tasks, and further optimize training efficiency.

Societal Impacts. Training and fine-tuning large deep models require substantial computational infrastructure, especially GPU resources, which consume energy and contribute to the environmental footprint of machine learning. By adopting such fine-tuning methods that can guarantee in- and out-of-distribution performance while also reducing the number of trainable parameters, the computational cost and the energy use associated with repeated model trainings can be reduced [19]. These methods provide a practical direction toward more sustainable use of large pretrained models.

References

- [1] Armen Aghajanyan, Luke Zettlemoyer, and Sonal Gupta. Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning, December 2020.
- [2] Shun-ichi Amari. Natural Gradient Works Efficiently in Learning. *Neural Computation*, 10(2): 251–276, February 1998. ISSN 0899-7667. doi: 10.1162/089976698300017746.
- [3] Shun-ichi Amari. *Information Geometry and Its Applications*. Springer, February 2016. ISBN 978-4-431-55978-8.
- [4] Alan Ansell, Edoardo Maria Ponti, Anna Korhonen, and Ivan Vulić. Composable Sparse Fine-Tuning for Cross-Lingual Transfer, February 2023.
- [5] Hyojin Bahng, Ali Jahanian, Swami Sankaranarayanan, and Phillip Isola. Exploring Visual Prompts for Adapting Large-Scale Models, June 2022.
- [6] Christian Benz and Volker Rodehorst. Omni-Crack30k: A Benchmark for Crack Segmentation and the Reasonable Effectiveness of Transfer Learning. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, pages 3876–3886, June 2024. doi: 10.1109/CVPRW63382.2024.00392.
- [7] Shoufa Chen, Chongjian Ge, Zhan Tong, Jiangliu Wang, Yibing Song, Jue Wang, and Ping Luo. AdaptFormer: Adapting Vision Transformers for Scalable Visual Recognition. *Advances in Neural Information Processing Systems*, 35:16664–16678, December 2022.
- [8] Zhe Chen, Yuchen Duan, Wenhai Wang, Junjun He, Tong Lu, Jifeng Dai, and Yu Qiao. Vision Transformer Adapter for Dense Predictions. In *The Eleventh International Conference on Learning Representations*, September 2022.
- [9] Zhuangzhuang Chen, Jin Zhang, Zhuonan Lai, Guanming Zhu, Zun Liu, Jie Chen, and Jianqiang Li. The Devil is in the Crack Orientation: A New Perspective for Crack Detection. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6630–6640, October 2023. doi: 10.1109/ICCV51070.2023.00612.
- [10] Zhuangzhuang Chen, Zhuonan Lai, Jie Chen, and Jianqiang Li. Mind marginal non-crack regions: Clustering-inspired representation learning for crack segmentation. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12698–12708, June 2024. doi: 10.1109/CVPR52733.2024.01207.

- [11] Felix Dangel, Bálint Mucsányi, Tobias Weber, and Runa Eschenhagen. Kronecker-factored Approximate Curvature (KFAC) From Scratch, July 2025.
- [12] Gintare Karolina Dziugaite and Daniel M Roy. Computing nonvacuous generalization bounds for deep (stochastic) neural networks with many more parameters than training data. *arXiv preprint arXiv:1703.11008*, 2017.
- [13] Jonathan Frankle and Michael Carbin. The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks, March 2019.
- [14] K. Ge, C. Wang, Y. T. Guo, Y. S. Tang, Z. Z. Hu, and H. B. Chen. Fine-tuning vision foundation model for crack segmentation in civil infrastructures. *Construction and Building Materials*, 431:136573, June 2024. ISSN 0950-0618. doi: 10.1016/j.conbuildmat.2024.136573.
- [15] Damien MARTINS Gomes, Yanlei Zhang, Eugene Belilovsky, Guy Wolf, and Mahdi S. Hosseini. AdaFisher: Adaptive Second Order Optimization via Fisher Information. In *The Thirteenth International Conference on Learning Representations*, October 2024.
- [16] Demi Guo, Alexander M. Rush, and Yoon Kim. Parameter-Efficient Transfer Learning with Diff Pruning, June 2021.
- [17] Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey, April 2024.
- [18] Xuehai He, Chunyuan Li, Pengchuan Zhang, Jianwei Yang, and Xin Eric Wang. Parameter-Efficient Model Adaptation for Vision Transformers. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(1):817–825, June 2023. ISSN 2374-3468. doi: 10.1609/aaai.v37i1.25160.
- [19] Sara Hooker. On the slow death of scaling. *Available at SSRN 5877662*, 2025.
- [20] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pages 2790–2799. PMLR, 2019.
- [21] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models, October 2021.
- [22] Qidong Huang, Xiaoyi Dong, Dongdong Chen, Weiming Zhang, Feifei Wang, Gang Hua, and Nenghai Yu. Diversity-Aware Meta Visual Prompting. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10878–10887, Vancouver, BC, Canada, June 2023. IEEE. ISBN 979-8-3503-0129-8. doi: 10.1109/CVPR52729.2023.01047.
- [23] Menglin Jia, Luming Tang, Bor-Chun Chen, Claire Cardie, Serge Belongie, Bharath Hariharan, and Ser-Nam Lim. Visual Prompt Tuning, July 2022.
- [24] Shibo Jie, Zhi-Hong Deng, Shixuan Chen, and Zhijuan Jin. Convolutional Bypasses Are Better Vision Transformer Adapters. In *ECAI 2024*, pages 202–209. IOS Press, 2024. doi: 10.3233/FAIA240489.
- [25] Minyoung Kim and Timothy Hospedales. BayesTune: Bayesian Sparse Deep Model Fine-tuning. *Advances in Neural Information Processing Systems*, 36:65317–65365, December 2023.
- [26] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017.
- [27] Ananya Kumar, Aditi Raghunathan, Robbie Matthew Jones, Tengyu Ma, and Percy Liang. Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution. In *International Conference on Learning Representations*, October 2021.

- [28] Yongshang Li, Ronggui Ma, Han Liu, and Gaoli Cheng. Real-time high-resolution neural network with semantic guidance for crack segmentation. *Automation in Construction*, 156: 105112, December 2023. ISSN 0926-5805. doi: 10.1016/j.autcon.2023.105112.
- [29] Haowei Lin, Baizhou Huang, Haotian Ye, Qinyu Chen, Zihao Wang, Sujian Li, Jianzhu Ma, Xiaojun Wan, James Zou, and Yitao Liang. Selecting Large Language Model to Fine-tune via Rectified Scaling Law. In *Proceedings of the 41st International Conference on Machine Learning*, pages 30080–30107. PMLR, July 2024.
- [30] Yuhao Liu, Saurabh Agarwal, and Shivaram Venkataraman. AutoFreeze: Automatically Freezing Model Blocks to Accelerate Fine-tuning, April 2021.
- [31] Sanae Lotfi, Marc Anton Finzi, Sanyam Kapoor, Andres Potapczynski, Micah Goldblum, and Andrew Gordon Wilson. PAC-Bayes compression bounds so tight that they can explain generalization. In *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=o8nYuR8ekFm>.
- [32] James Martens and Roger Grosse. Optimizing Neural Networks with Kronecker-factored Approximate Curvature. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2408–2417. PMLR, June 2015.
- [33] David A. McAllester. PAC-Bayesian model averaging. In *Proceedings of the Twelfth Annual Conference on Computational Learning Theory*, pages 164–170, Santa Cruz California USA, July 1999. ACM. ISBN 978-1-58113-167-3. doi: 10.1145/307400.307435.
- [34] David A. McAllester. Some PAC-Bayesian Theorems. *Machine Learning*, 37(3):355–363, December 1999. ISSN 1573-0565. doi: 10.1023/A:1007618624809.
- [35] Peng Mi, Li Shen, Tianhe Ren, Yiyi Zhou, Xiaoshuai Sun, Rongrong Ji, and Dacheng Tao. Make Sharpness-Aware Minimization Stronger: A Sparsified Perturbation Approach. *Advances in Neural Information Processing Systems*, 35:30950–30962, December 2022.
- [36] Namuk Park and Songkuk Kim. How Do Vision Transformers Work? In *International Conference on Learning Representations*, October 2021.
- [37] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, Eric Mintun, Junting Pan, Kalyan Vasudev Alwala, Nicolas Carion, Chao-Yuan Wu, Ross Girshick, Piotr Dollár, and Christoph Feichtenhofer. SAM 2: Segment Anything in Images and Videos, October 2024.
- [38] Alexander Soen and Ke Sun. Trade-Offs of Diagonal Fisher Information Matrix Estimators. *Advances in Neural Information Processing Systems*, 37:5870–5912, December 2024.
- [39] Yi-Lin Sung, Varun Nair, and Colin Raffel. Training Neural Networks with Fixed Sparse Masks - Preprint, November 2021.
- [40] Yi Tay, Mostafa Dehghani, Jinfeng Rao, William Fedus, Samira Abnar, Hyung Won Chung, Sharan Narang, Dani Yogatama, Ashish Vaswani, and Donald Metzler. Scale Efficiently: Insights from Pretraining and Finetuning Transformers. In *International Conference on Learning Representations*, October 2021.
- [41] Wenjun Wang and Chao Su. Automatic concrete crack segmentation model based on transformer. *Automation in Construction*, 139:104275, July 2022. ISSN 09265805. doi: 10.1016/j.autcon.2022.104275.
- [42] Zhengbo Wang, Jian Liang, Ran He, Zilei Wang, and Tieniu Tan. LoRA-Pro: Are Low-Rank Adapters Properly Optimized? In *The Thirteenth International Conference on Learning Representations*, October 2024.
- [43] Enze Xie, Wenhao Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. In *Advances in Neural Information Processing Systems*, volume 34, pages 12077–12090. Curran Associates, Inc., 2021.

- [44] Runxin Xu, Fuli Luo, Zhiyuan Zhang, Chuanqi Tan, Baobao Chang, Songfang Huang, and Fei Huang. Raise a Child in Large Language Model: Towards Effective and Generalizable Fine-tuning, September 2021.
- [45] Elad Ben Zaken, Shauli Ravfogel, and Yoav Goldberg. BitFit: Simple Parameter-efficient Fine-tuning for Transformer-based Masked Language-models, September 2022.
- [46] Dan Zhang, Tao Feng, Lilong Xue, Yuandong Wang, Yuxiao Dong, and Jie Tang. Parameter-Efficient Fine-Tuning for Foundation Models, January 2025.
- [47] Qingru Zhang, Minshuo Chen, Alexander Bukharin, Nikos Karampatziakis, Pengcheng He, Yu Cheng, Weizhu Chen, and Tuo Zhao. Adalora: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint arXiv:2303.10512*, 2023.
- [48] Bingchen Zhao, Haoqin Tu, Chen Wei, Jieru Mei, and Cihang Xie. Tuning LayerNorm in Attention: Towards Efficient Multi-Modal LLM Finetuning. In *The Twelfth International Conference on Learning Representations*, October 2023.
- [49] Dong Zhao, Jinlong Li, Shuang Wang, Mengyao Wu, Qi Zang, Nicu Sebe, and Zhun Zhong. FisherTune: Fisher-Guided Robust Tuning of Vision Foundation Models for Domain Generalized Segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15043–15054, 2025.
- [50] Mengjie Zhao, Tao Lin, Fei Mi, Martin Jaggi, and Hinrich Schütze. Masking as an Efficient Alternative to Finetuning for Pretrained Language Models, October 2020.
- [51] Wenda Zhou, Victor Veitch, Morgane Austern, Ryan P Adams, and Peter Orbanz. Non-vacuous generalization bounds at the imagenet scale: a pac-bayesian compression approach. *arXiv preprint arXiv:1804.05862*, 2018.
- [52] Fuzhen Zhuang, Zhiyuan Qi, Keyu Duan, Dongbo Xi, Yongchun Zhu, Hengshu Zhu, Hui Xiong, and Qing He. A Comprehensive Survey on Transfer Learning. *Proceedings of the IEEE*, 109(1):43–76, January 2021. ISSN 1558-2256. doi: 10.1109/JPROC.2020.3004555.

Appendix Overview

The appendix provides theoretical derivations, implementation details for Fisher drift, supplementary experimental results, and the full FisherAdapTune algorithm. Detailed threshold ablation tables are reported in Appendix B.2.

A Theoretical Details

A.1 Proof of Proposition 3.2 (Eq. (3)):

$$D_{\text{KL}}(Q_T \| P) = \sum_{t=0}^{T-1} \left[\frac{1}{2} \delta_t^\top \mathbf{F}_{\theta_0} \delta_t + \delta_t^\top \mathbf{F}_{\theta_0} (\theta_t - \theta_0) \right].$$

Step 1: KL between two Gaussians with the same covariance. Since $Q_T = \mathcal{N}(\theta_T, \mathbf{F}_{\theta_0}^{-1})$ and $P = \mathcal{N}(\theta_0, \mathbf{F}_{\theta_0}^{-1})$ share the same covariance, the standard KL formula for multivariate Gaussians reduces to a pure quadratic in the difference of means (all log-determinant and trace terms cancel):

$$D_{\text{KL}}(Q_T \| P) = \frac{1}{2} (\theta_T - \theta_0)^\top \mathbf{F}_{\theta_0} (\theta_T - \theta_0) = \frac{1}{2} \|\theta_T - \theta_0\|_{\mathbf{F}_{\theta_0}}^2.$$

Where the $\|\cdot\|_{\mathbf{F}_{\theta_0}}$ is the squared Mahalanobis norm weighted by the shared precision matrix. The distance measures how far the fine-tuned parameters θ_T have drifted from the pretrained θ_0 in Fisher-geometry terms.

Step 2: Expressing the total displacement as a telescoping sum. Define the per-step update $\delta_t = \theta_{t+1} - \theta_t$. Summing over all steps:

$$\theta_T - \theta_0 = \sum_{t=0}^{T-1} \delta_t.$$

Substituting into Step 1:

$$D_{\text{KL}}(Q_T \| P) = \frac{1}{2} \left\| \sum_{t=0}^{T-1} \delta_t \right\|_{\mathbf{F}_{\theta_0}}^2.$$

Step 3: Expanding the squared norm via a telescoping identity. Let $\mathbf{F}_0 \equiv \mathbf{F}_{\theta_0}$ for brevity. Define the partial-sum vector $\mathbf{s}_t = \sum_{s=0}^{t-1} \delta_s = \theta_t - \theta_0$, with $\mathbf{s}_0 = \mathbf{0}$ and $\mathbf{s}_T = \theta_T - \theta_0$. Then:

$$\frac{1}{2} \|\mathbf{s}_T\|_{\mathbf{F}_0}^2 = \sum_{t=0}^{T-1} \left(\frac{1}{2} \|\mathbf{s}_{t+1}\|_{\mathbf{F}_0}^2 - \frac{1}{2} \|\mathbf{s}_t\|_{\mathbf{F}_0}^2 \right).$$

Since $\mathbf{s}_{t+1} = \mathbf{s}_t + \delta_t$, each difference expands as:

$$\begin{aligned} \frac{1}{2} \|\mathbf{s}_t + \delta_t\|_{\mathbf{F}_0}^2 - \frac{1}{2} \|\mathbf{s}_t\|_{\mathbf{F}_0}^2 &= \frac{1}{2} (\mathbf{s}_t^\top \mathbf{F}_0 \mathbf{s}_t + 2 \mathbf{s}_t^\top \mathbf{F}_0 \delta_t + \delta_t^\top \mathbf{F}_0 \delta_t) - \frac{1}{2} \mathbf{s}_t^\top \mathbf{F}_0 \mathbf{s}_t \\ &= \frac{1}{2} \delta_t^\top \mathbf{F}_0 \delta_t + \delta_t^\top \mathbf{F}_0 \mathbf{s}_t. \end{aligned}$$

Substituting $\mathbf{s}_t = \theta_t - \theta_0$:

$$\frac{1}{2} \|\mathbf{s}_{t+1}\|_{\mathbf{F}_0}^2 - \frac{1}{2} \|\mathbf{s}_t\|_{\mathbf{F}_0}^2 = \frac{1}{2} \delta_t^\top \mathbf{F}_0 \delta_t + \delta_t^\top \mathbf{F}_0 (\theta_t - \theta_0).$$

Step 4: Summing over all steps. Summing from $t = 0$ to $T - 1$ and using the result of Step 1:

$$D_{\text{KL}}(Q_T \| P) = \frac{1}{2} \|\mathbf{s}_T\|_{\mathbf{F}_0}^2 = \sum_{t=0}^{T-1} \left[\frac{1}{2} \delta_t^\top \mathbf{F}_0 \delta_t + \delta_t^\top \mathbf{F}_0 (\theta_t - \theta_0) \right],$$

which gives Eq. (3).

A.2 Jensen-Shannon Distance for Fisher Structural Drift

To compare Fisher information across layers or training stages, for each layer ℓ , the Fisher values of weights and biases are concatenated into a single vector:

$$F_\ell = \{f_i\}_{i=1}^{N_\ell}. \quad (10)$$

To stabilize heavy-tailed magnitudes, we operate in log-scale:

$$\tilde{F}_\ell = \log_{10}(|F_\ell| + \varepsilon), \quad \varepsilon = 10^{-12}. \quad (11)$$

To compare two Fisher states, we construct histograms using a shared set of B bins. The bin edges $\{b_k\}_{k=0}^B$ are computed from the joint percentile range (1%-99%) of the two log-transformed arrays being compared. The histogram counts for layer ℓ are computed as:

$$c_k = \#\left\{i \mid b_k \leq \tilde{f}_i < b_{k+1}\right\}, \quad k = 0, \dots, B-1, \quad (12)$$

where $\tilde{f}_i \in \tilde{F}_\ell$.

To avoid zero-probability bins and ensure numerical stability in divergence computations, we apply additive smoothing and normalize:

$$p_k = \frac{c_k + \varepsilon}{\sum_{j=0}^{B-1} (c_j + \varepsilon)}, \quad \sum_{k=0}^{B-1} p_k = 1. \quad (13)$$

The resulting probability mass function (PMF) for layer ℓ is

$$p_\ell = (p_0, \dots, p_{B-1}) \in \mathbb{R}^B. \quad (14)$$

Distributional differences are then measured using the Jensen-Shannon (JS) distance. The JS divergence between two PMFs p and q is defined as:

$$\text{JS}(p, q) = \frac{1}{2} \text{KL}(p \parallel m) + \frac{1}{2} \text{KL}(q \parallel m), \quad m = \frac{1}{2}(p + q). \quad (15)$$

And the JS distance is:

$$d_{\text{JS}}(p, q) = \sqrt{\text{JS}(p, q)}. \quad (16)$$

This symmetric, bounded distance provides a stable measure of layer-wise Fisher redistribution during fine-tuning.

B Additional Experimental Results

All fine-tuning experiments were conducted using AdamW with a cosine learning-rate schedule and a batch size of 2. We used a learning rate of 10^{-6} for SAM2 and 10^{-4} for SegFormer-based models. Training optimizes a binary crack-segmentation objective combining BCEWithLogits loss with Dice loss, and all images and masks were resized to 256×256 for training and evaluation. FisherAdapTune experiments used the same optimizer, loss, and data splits; the only difference was that parameter groups were progressively removed from the optimizer when their smoothed Fisher JS drift falls below the adaptive freezing criterion. All experiments were conducted on the Digital Research Alliance of Canada (the Alliance) infrastructure using NVIDIA H100 and local NVIDIA RTX 6000 Ada Generation GPUs.

B.1 Fisher Drift Observations

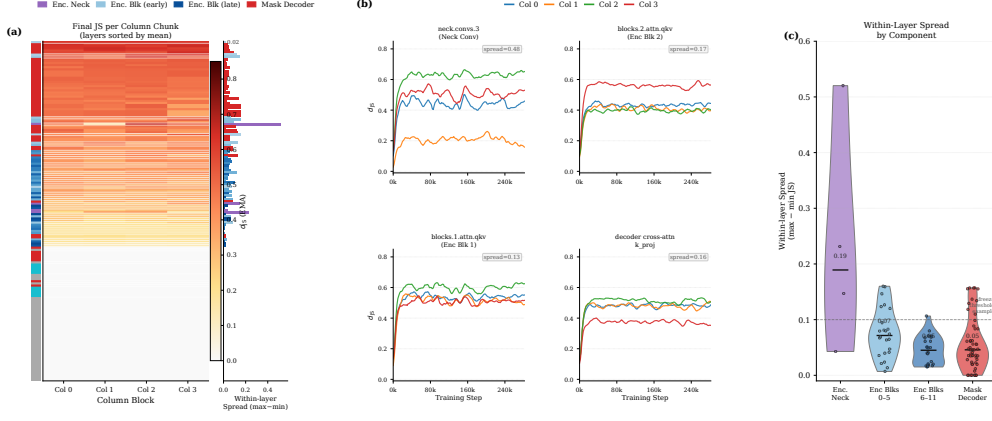


Figure 7: Parameter group-wise Jensen-Shannon (JS) distance between consecutive Fisher distributions in SAM2-Tiny during full fine-tuning on crack segmentation; $EMA[d_{JS_i}^t(p(\tilde{\mathbf{F}}_i^{t-1}), p(\tilde{\mathbf{F}}_i^t))]$.

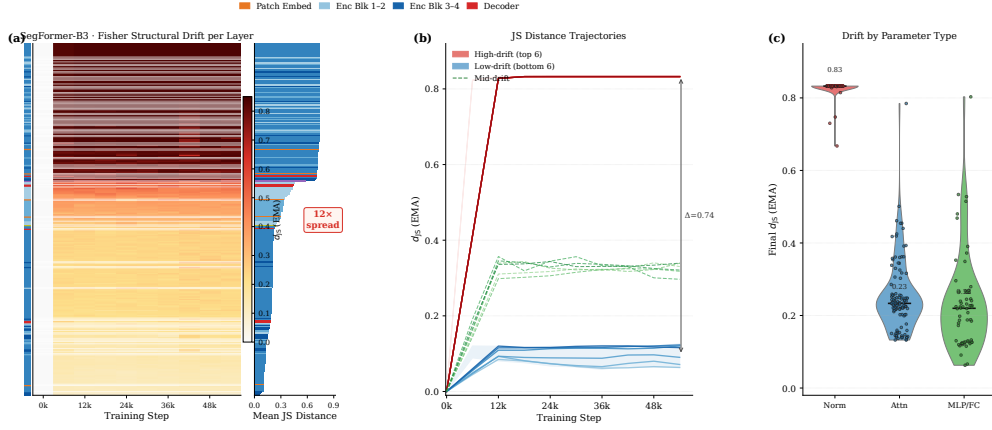


Figure 8: Layer-wise Jensen-Shannon (JS) distance between consecutive Fisher distributions in SegFormer during full fine-tuning on crack segmentation; $EMA[d_{JS_i}^t(p(\tilde{\mathbf{F}}_i^{t-1}), p(\tilde{\mathbf{F}}_i^t))]$.

B.2 Detailed Threshold Ablations

Tables 3 and 4 report the full threshold ablation used to select λ . The final trainable-parameter count is the active set at the end of training, while the average count reflects the average effective number of trainable parameters. The optimal m (Fisher collection interval) and n (freezing interval) were 300 and 3000, respectively, in all experiments.

Table 3: Ablation on the freezing threshold λ for **FisherAdapTune** applied to SAM2-Tiny and SAM2-Large. Larger $|\lambda|$ retains more trainable parameter groups. Bold entries show the best per column within each model block; underline shows second best. Shaded rows indicate the selected λ .

Model	λ	Final Tunable Params	Avg. Tunable Params	OmniCrack		Zero-shot F1 (%)				Zero-shot IoU (%)			
				F1	IoU	Conc.	Fac.	Road	Avg. $\pm$ Std	Conc.	Fac.	Road	Avg. $\pm$ Std
SAM2-Tiny	0.0	4.7K	3.82M	61.96	44.89	72.77	69.65	66.49	69.64$\pm$3.14	57.20	53.43	49.80	53.48$\pm$3.70
	-0.5	4.7K	3.08M	61.42	44.32	71.87	<u>69.22</u>	65.85	68.98 $\pm$ 3.02	56.09	<u>52.93</u>	49.09	52.70 $\pm$ 3.51
	-0.8	0.20M	5.40M	62.40	45.35	72.51	68.58	<u>66.29</u>	<u>69.13$\pm$3.15</u>	56.88	52.19	<u>49.58</u>	<u>52.88$\pm$3.70</u>
	-0.85	0.27M	6.95M	63.12	46.11	72.43	69.10	64.42	<u>68.65$\pm$4.0</u>	56.77	52.79	47.52	<u>52.36$\pm$4.6</u>
	-0.87	9.9M	12.13M	64.20	47.27	<u>72.98</u>	68.84	64.98	68.93 $\pm$ 4.00	<u>57.45</u>	52.48	48.14	52.69 $\pm$ 4.67
	-0.9	11.8M	13.69M	64.28	47.36	72.99	68.74	64.99	68.91 $\pm$ 4.00	57.47	52.37	48.14	52.66 $\pm$ 4.67
	-0.95	13.1M	15.02M	<u>64.41</u>	<u>47.50</u>	72.92	68.99	64.97	68.96 $\pm$ 3.98	57.39	52.67	48.11	52.72 $\pm$ 4.64
	-1	16.6M	18.07M	64.51	47.61	72.94	69.05	64.97	68.98 $\pm$ 3.99	57.40	52.73	48.11	52.75 $\pm$ 4.65
SAM2-Large	0.0	7.1K	17.41M	57.43	40.29	65.17	73.55	66.44	68.39 $\pm$ 3.69	48.33	58.17	49.74	52.08 $\pm$ 4.34
	-0.5	7.1K	25.62M	59.60	42.45	68.94	73.93	67.63	70.17 $\pm$ 2.72	52.60	58.64	51.09	54.11 $\pm$ 3.26
	-0.7	1.3M	34.36M	62.77	45.74	72.63	74.10	68.58	71.77 $\pm$ 2.33	57.02	58.85	52.18	56.02 $\pm$ 2.81
	-0.85	2.0M	59.40M	64.57	47.67	71.99	74.69	68.04	71.57 $\pm$ 2.73	56.25	59.61	51.56	55.80 $\pm$ 3.30
	-0.86	2.1M	55.36M	64.33	47.41	71.81	75.79	68.54	72.05 $\pm$ 2.96	56.02	61.01	52.14	56.39 $\pm$ 3.63
	-0.87	3.0M	71.49M	<u>65.78</u>	<u>49.01</u>	72.81	<u>75.30</u>	70.29	<u>72.80$\pm$2.04</u>	57.24	60.38	54.19	<u>57.27$\pm$2.53</u>
	-0.88	2.5M	63.52M	65.38	48.57	72.17	74.90	69.33	72.13 $\pm$ 2.28	56.46	59.87	53.05	56.46 $\pm$ 2.78
	-0.9	56.8M	82.54M	64.72	47.84	73.32	74.72	71.27	73.10$\pm$1.42	57.88	59.64	55.36	57.63$\pm$1.76
	-0.95	74.5M	97.12M	65.81	49.04	<u>73.13</u>	73.89	69.24	72.08 $\pm$ 2.03	<u>57.64</u>	58.59	52.95	56.39 $\pm$ 2.46
	-1.0	81.9M	101.14M	65.56	48.77	72.10	<u>75.37</u>	<u>70.60</u>	72.69 $\pm$ 1.99	56.38	<u>60.48</u>	<u>54.56</u>	<u>57.14$\pm$2.48</u>

Table 4: Ablation on the freezing threshold λ for **FisherAdapTune** applied to SegFormer-MiT-B0 and SegFormer-MiT-B3. Larger $|\lambda|$ retains more trainable parameter groups. Bold shows best per column within each model block; underline shows second best. Shaded rows indicate the selected λ .

Model	λ	Final Tunable Params	Avg. Tunable Params	OmniCrack		Zero-shot F1 (%)				Zero-shot IoU (%)			
				F1	IoU	Conc.	Fac.	Road	Avg. $\pm$ Std	Conc.	Fac.	Road	Avg. $\pm$ Std
SegFormer-B0	-0.50	0.032K	0.047M	58.44	41.28	70.18	58.70	52.30	60.39 $\pm$ 7.39	54.06	41.54	35.41	43.66 $\pm$ 7.76
	-0.65	0.032K	0.074M	59.53	42.37	71.40	<u>59.17</u>	52.03	60.86 $\pm$ 7.99	55.52	<u>42.02</u>	35.17	44.23 $\pm$ 8.45
	-0.75	0.032K	0.085M	59.65	42.50	71.36	59.92	56.06	62.44$\pm$6.49	55.47	42.77	38.95	45.73$\pm$7.06
	-0.80	0.064K	0.089M	60.16	43.02	70.71	58.99	54.99	<u>61.56$\pm$6.67</u>	54.70	41.83	<u>37.92</u>	44.81 $\pm$ 7.16
	-0.85	0.064K	0.093M	60.23	43.09	69.66	58.54	54.52	<u>60.90$\pm$6.40</u>	53.44	41.38	37.48	44.09 $\pm$ 6.79
	-0.90	0.096K	0.100M	60.09	42.95	68.77	58.50	52.33	59.86 $\pm$ 6.78	52.41	41.35	35.43	43.06 $\pm$ 7.03
	-0.95	0.128K	0.124M	60.85	43.73	70.97	58.77	53.63	<u>61.12$\pm$7.27</u>	55.01	41.62	36.64	<u>44.42$\pm$7.75</u>
	-1.00	0.160K	0.144M	60.79	43.67	70.62	59.02	53.29	60.97 $\pm$ 7.20	54.59	41.87	36.32	44.26 $\pm$ 7.64
	-1.25	0.048M	0.360M	62.28	45.22	70.52	58.89	52.59	60.67 $\pm$ 7.43	54.46	41.73	35.67	43.96 $\pm$ 7.83
	-1.5	1.31M	1.85M	62.96	45.95	73.03	56.58	<u>54.98</u>	61.53 $\pm$ 8.16	57.52	39.45	37.91	44.96 $\pm$ 8.90
	-2	3.37M	3.54M	<u>63.23</u>	<u>46.23</u>	<u>73.02</u>	58.01	53.64	<u>61.56$\pm$8.30</u>	<u>57.50</u>	40.85	36.65	45.00 $\pm$ 9.00
SegFormer-B3	-3	3.71M	3.71M	63.25	46.25	72.76	56.96	53.56	61.09 $\pm$ 8.36	57.18	39.82	36.57	44.52 $\pm$ 9.05
	-0.50	3K	5.02M	61.84	44.76	73.21	<u>60.80</u>	52.66	62.22 $\pm$ 8.44	57.74	<u>43.68</u>	35.74	45.72 $\pm$ 9.09
	-0.65	4K	6.19M	62.83	45.81	72.90	60.73	51.72	61.78 $\pm$ 8.67	57.36	43.61	34.88	45.28 $\pm$ 9.25
	-0.75	6K	7.10M	62.94	45.92	73.12	60.19	53.45	62.25 $\pm$ 8.16	57.63	43.05	36.47	45.71 $\pm$ 8.84
	-0.80	7K	7.80M	63.76	46.80	73.64	59.21	53.38	62.07 $\pm$ 8.51	58.29	42.06	36.40	45.58 $\pm$ 9.27
	-0.85	7K	8.48M	63.54	46.57	73.04	59.89	55.38	62.77 $\pm$ 7.49	57.53	42.74	38.30	46.19 $\pm$ 8.22
	-0.90	7K	9.38M	63.90	46.95	73.35	59.39	<u>57.76</u>	63.50 $\pm$ 6.99	57.91	42.24	<u>40.60</u>	46.91 $\pm$ 7.80
	-0.95	11K	10.73M	<u>64.57</u>	<u>47.68</u>	<u>74.19</u>	61.04	58.26	64.49$\pm$6.94	<u>58.97</u>	43.93	41.11	48.00$\pm$7.83
	-1.00	14K	12.81M	64.97	48.11	75.85	57.83	56.52	63.40 $\pm$ 8.81	61.10	40.68	39.40	47.06 $\pm$ 9.94

B.3 Additional Ablation Visualizations

The following plots show how the active parameter budget evolves during training for each threshold. Smaller $|\lambda|$ values freeze parameter groups earlier, producing steep budget decay, while more conservative thresholds keep a larger active set for longer.

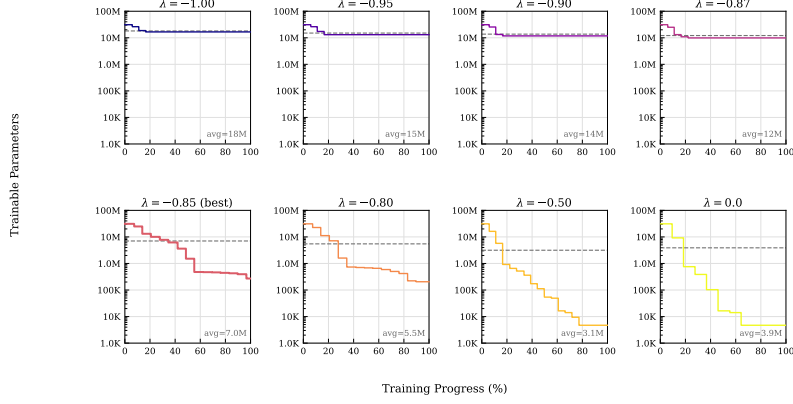


Figure 9: Trainable-parameter evolution for SAM2-Tiny across freezing thresholds λ .

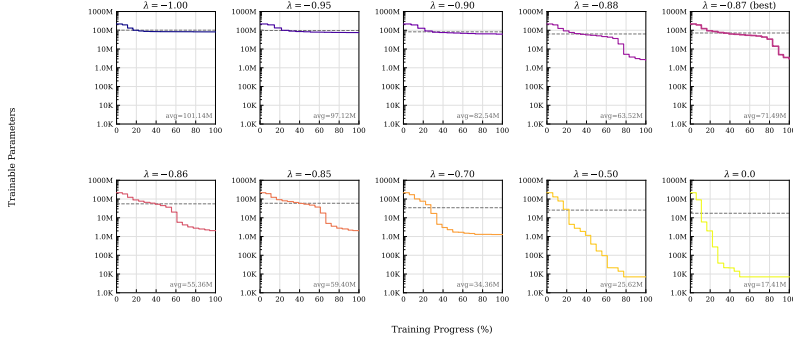


Figure 10: Trainable-parameter evolution for SAM2-Large across freezing thresholds λ .

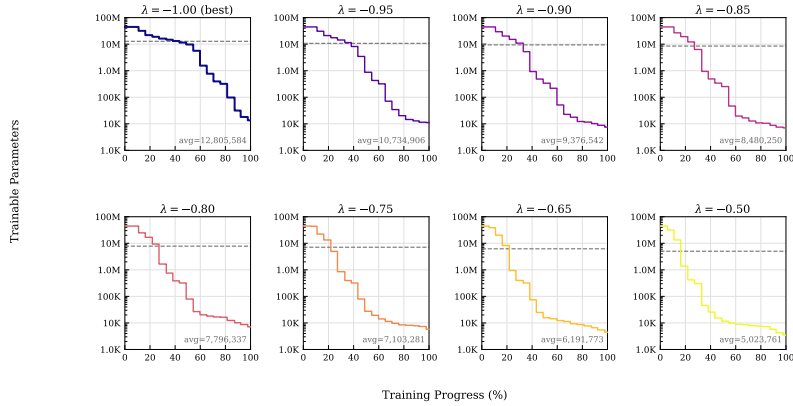


Figure 11: Trainable-parameter evolution for SegFormer-MiT-B3 across freezing thresholds λ .

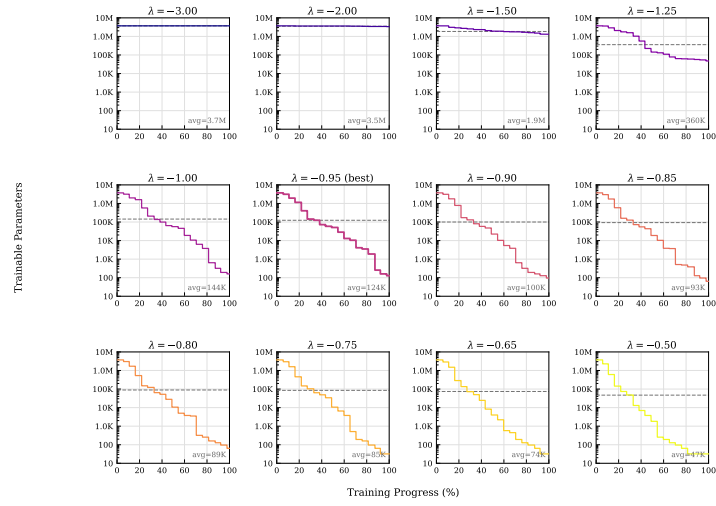


Figure 12: Trainable-parameter evolution for SegFormer-MiT-B0 across freezing thresholds λ .

C Detailed FisherAdapTune Algorithm

Algorithm 2 expands the main-text Algorithm 1. The active set $\mathcal{S}$ contains the parameter groups that remain trainable; frozen groups are masked out of the optimizer update.

Algorithm 2 FisherAdapTune Algorithm

Require: Parameters $\theta = \{\theta_i\}_{i=1}^L$, dataset $\mathcal{D}$, EMA factors γ, β , threshold scale λ , slice count k , Fisher interval m , freeze interval n with $m < n$, total steps $T_{\max}$.

1: Initialize active parameter groups set $\mathcal{S}$ to all supported groups.

2: **for** $t = 1$ **to** $T_{\max}$ **do**

3: Compute loss $\mathcal{L}(\mathcal{D}; \theta)$ and gradients $\nabla_{\theta} \mathcal{L}$.

4: **if** $t \bmod m = 0$ **then**

5: Update AdaFisher activation and gradient statistics:

$$\mathbf{H}_{D_{i-1}}^t \leftarrow \gamma \mathbf{H}_{D_{i-1}}^{t-1} + (1 - \gamma) \hat{\mathbf{H}}_{D_{i-1}}^t, \quad \mathbf{S}_{D_i}^t \leftarrow \gamma \mathbf{S}_{D_i}^{t-1} + (1 - \gamma) \hat{\mathbf{S}}_{D_i}^t.$$

6: Estimate layer Fisher tensors $\tilde{\mathbf{F}}_{D_i}^t = \mathbf{H}_{D_{i-1}}^t \otimes \mathbf{S}_{D_i}^t$.

7: Partition each $\tilde{\mathbf{F}}_{D_i}^t$ into k column-wise parameter groups $\{\tilde{\mathbf{F}}_{w_{ik}}^t\}_k$.

8: Build PMFs $p_{ik}^t = p(\tilde{\mathbf{F}}_{w_{ik}}^t)$ using Eqs. (11)-(14).

9: Compute consecutive Fisher drift:

$$\tilde{d}_{\text{JS},ik}^t = d_{\text{JS}}(p_{ik}^{t-1}, p_{ik}^t) = \sqrt{\frac{1}{2} \text{KL}(p_{ik}^{t-1} \| M_{ik}^t) + \frac{1}{2} \text{KL}(p_{ik}^t \| M_{ik}^t)}, \quad M_{ik}^t = \frac{1}{2}(p_{ik}^{t-1} + p_{ik}^t).$$

10: Smooth and accumulate the drift: $\hat{d}_{\text{JS},ik}^t \leftarrow \beta \hat{d}_{\text{JS},ik}^{t-1} + (1 - \beta) \tilde{d}_{\text{JS},ik}^t$, $\mathcal{J}_{ik} \leftarrow \mathcal{J}_{ik} \cup \{\hat{d}_{\text{JS},ik}^t\}$.

11: Compute running mean $\bar{d}_{\text{JS},ik} \leftarrow |\mathcal{J}_{ik}|^{-1} \sum_{d \in \mathcal{J}_{ik}} d$.

12: **if** $t \bmod n = 0$ **then**

13: Compute global drift statistics over active parameter groups: $\mu \leftarrow \text{mean}_{(i,k) \in \mathcal{S}}(\bar{d}_{\text{JS},ik})$, $\sigma \leftarrow \text{std}_{(i,k) \in \mathcal{S}}(\bar{d}_{\text{JS},ik})$.

14: Set $\tau \leftarrow \mu + \lambda \sigma$ and freeze stabilized parameter groups: $\mathcal{S} \leftarrow \{(i, k) \in \mathcal{S} : \bar{d}_{\text{JS},ik} \geq \tau\}$.

15: Update only active parameter groups: $\theta_{\mathcal{S}} \leftarrow \theta_{\mathcal{S}} - \eta \nabla_{\theta_{\mathcal{S}}} \mathcal{L}(\mathcal{D}; \theta)$.

16: **end for**
